



Al-Imam University

Computer Science Department

College of Computer and Information Sciences

Design and Implementation of a Compiler Front-End for the HealthScript Language

Phase 3

Students:

Name	ID
Nuha Aedh Alrubayyi	444005638
Dan Riyadh Alhumrani	444006468

Table of Contents

No.	Title	Page No.
1	Phase 3 Objective	3
2	Enhanced Symbol Table	3
3	Semantic Rules	4
4	Semantic Error Handling	5
5	Code Generation Approach	5
6	Test Cases	5
7	End-to-End Compilation	10
8	Appendix	11
9	Conclusion	16
10	References	18

1. Phase 3 Objective

The objective of Phase 3 is to extend the HealthScript compiler front-end by adding semantic analysis and code generation. The semantic analyzer checks whether the program is logically meaningful after passing lexical and syntax analysis. It verifies declarations, duplicate variables, type compatibility, and semantic errors. After successful semantic analysis, the system generates Three-Address Code (TAC) as an intermediate representation.

2. Enhanced Symbol Table

In Phase 3, the symbol table was extended to store additional information about identifiers. Instead of only storing the identifier name and first occurrence line, the enhanced symbol table stores the identifier name, data type, scope level, and declaration line.

The enhanced symbol table supports semantic checking by allowing the compiler to verify whether variables are declared before use, whether duplicate declarations exist, and whether assignments match the declared variable types.

Field	Description
Identifier Name	Name of the variable
Type	int, float, string, bool
Scope	global
Declaration Line	Line where identifier was declared

3. Semantic Rules

The semantic analyzer applies the following rules:

1. Every variable must be declared before use.
2. A variable cannot be declared more than once in the same scope.
3. The assigned value must be compatible with the declared variable type.
4. Integer values can be assigned to float variables.
5. String values cannot be assigned to int or float variables.
6. Arithmetic operations are only allowed between numeric types.
7. Division by zero is reported when the divisor is a literal zero.
8. Print statements must reference valid declared identifiers or valid literals.

Semantic Inference Rules

4. Semantic Error Handling

The semantic analyzer reports errors with clear messages and line numbers. Detected semantic errors include undeclared variables, duplicate declarations, incompatible assignments, invalid arithmetic operations, and division by zero.

5. Code Generation Approach

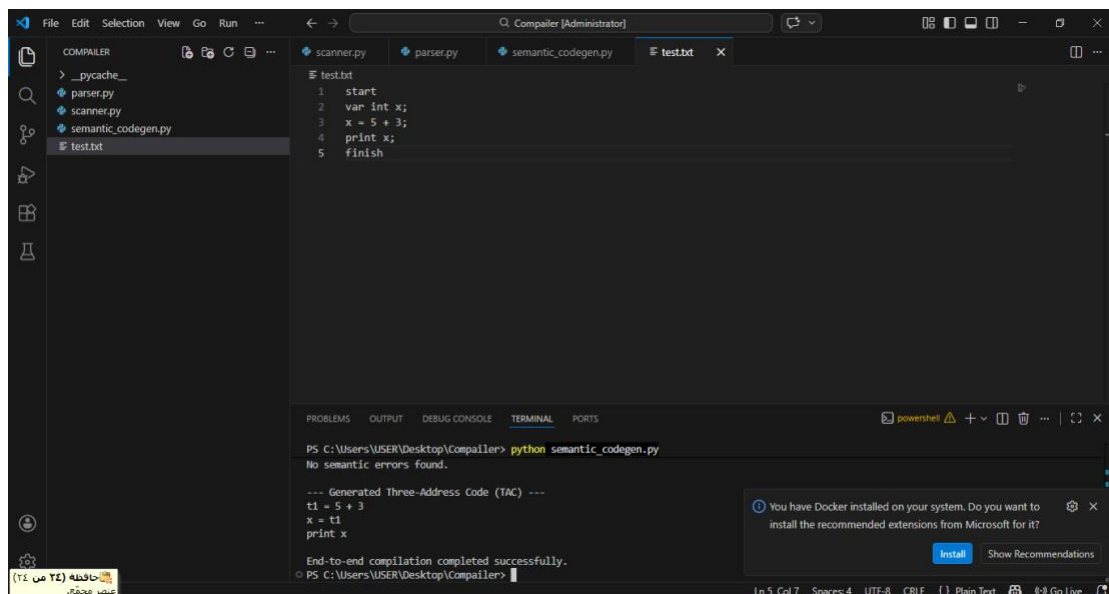
For code generation, the project uses Three-Address Code (TAC). TAC was selected because it is simple, readable, and suitable for educational compiler projects. Each TAC instruction contains at most one operation, which makes it easy to analyze and interpret.

6. Test Cases

Test Case 1: Correct Program with TAC Generation Code

Code

```
start
var int x;
x = 5 + 3;
print x;
finish
```



The screenshot shows the Compiler IDE interface. The editor displays the test program code. The terminal output shows the command `python semantic_codegen.py` being executed, resulting in the following output:

```
PS C:\Users\USER\Desktop\Compiler> python semantic_codegen.py
No semantic errors found.

--- Generated Three-Address Code (TAC) ---
t1 = 5 + 3
x = t1
print x

End-to-end compilation completed successfully.
PS C:\Users\USER\Desktop\Compiler>
```

A notification box in the bottom right corner states: "You have Docker installed on your system. Do you want to install the recommended extensions from Microsoft for it?" with buttons for "Install" and "Show Recommendations".

Output

Parsing successful.

Semantic analysis completed successfully.

Generated Three-Address Code (TAC):

```
t1 = 5 + 3
```

```
x = t1
```

```
print x
```

End-to-end compilation completed successfully.

Test Case 2: Type Mismatch Error

Code:

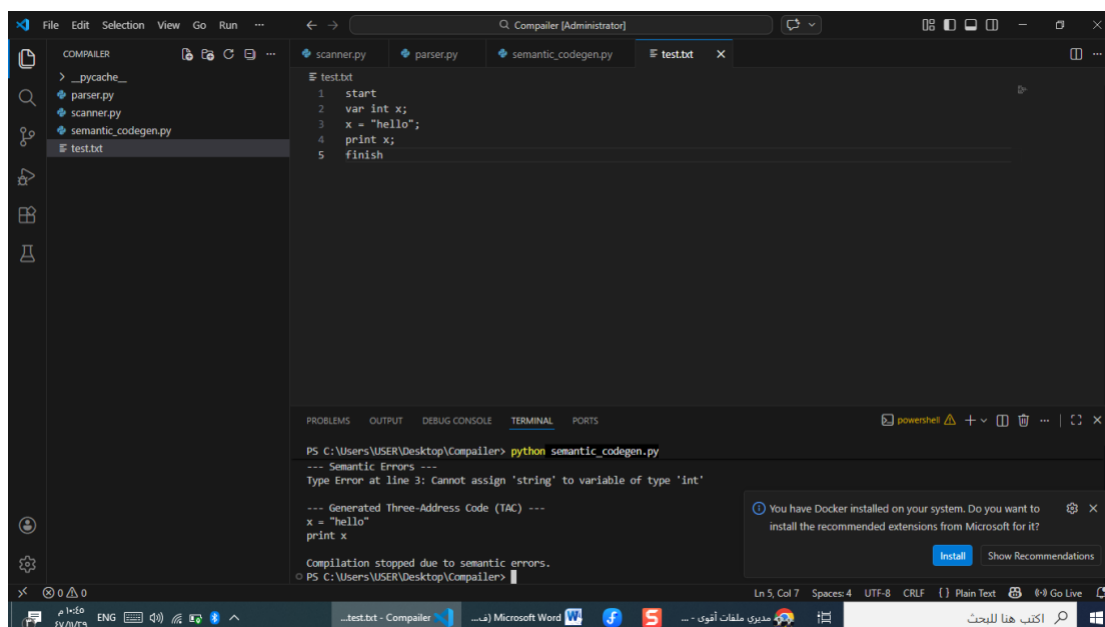
```
start
```

```
var int x;
```

```
x = "hello";
```

```
print x;
```

```
finish
```



Output:

Type Error at line 3: Cannot assign 'string' to variable of type 'int'

Compilation stopped due to semantic errors.

Test Case 3: Undeclared Variable Error

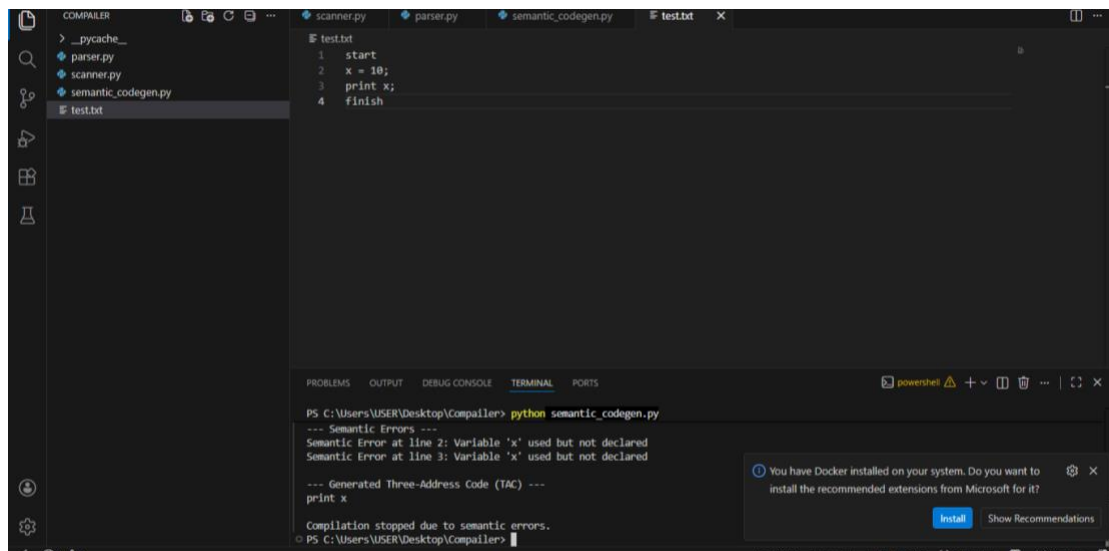
Code:

start

x = 10;

print x;

finish



Output:

Semantic Error at line 2: Variable 'x' used but not declared

Semantic Error at line 3: Variable 'x' used but not declared

Test Case 4 — Duplicate Declaration Error

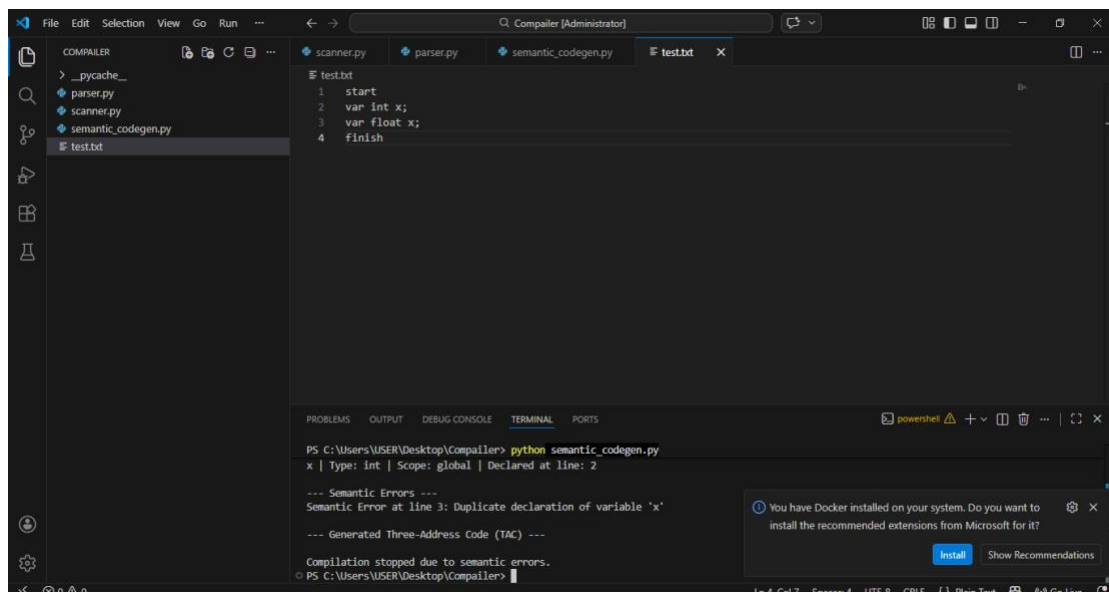
Code:

start

var int x;

var float x;

finish



The screenshot shows a Visual Studio Code editor window titled 'Compiler [Administrator]'. The file explorer on the left shows a project named 'COMPILER' with files: '_pycache_', 'parser.py', 'scanner.py', 'semantic_codegen.py', and 'test.txt'. The 'test.txt' file is open in the editor, containing the following code:

```
1 start
2 var int x;
3 var float x;
4 finish
```

The bottom panel shows the 'TERMINAL' tab with the following output:

```
PS C:\Users\USER\Desktop\Compiler> python semantic_codegen.py
x | Type: int | Scope: global | Declared at line: 2
--- Semantic Errors ---
Semantic Error at line 3: Duplicate declaration of variable 'x'
--- Generated Three-Address Code (TAC) ---
Compilation stopped due to semantic errors.
PS C:\Users\USER\Desktop\Compiler>
```

A notification bubble on the right side of the terminal says: 'You have Docker installed on your system. Do you want to install the recommended extensions from Microsoft for it?' with an 'Install' button and a 'Show Recommendations' link.

Output:

Semantic Error at line 3: Duplicate declaration of variable 'x'

Compilation stopped due to semantic errors.

Test Case 5 — Correct Program with Float Operations

Code:

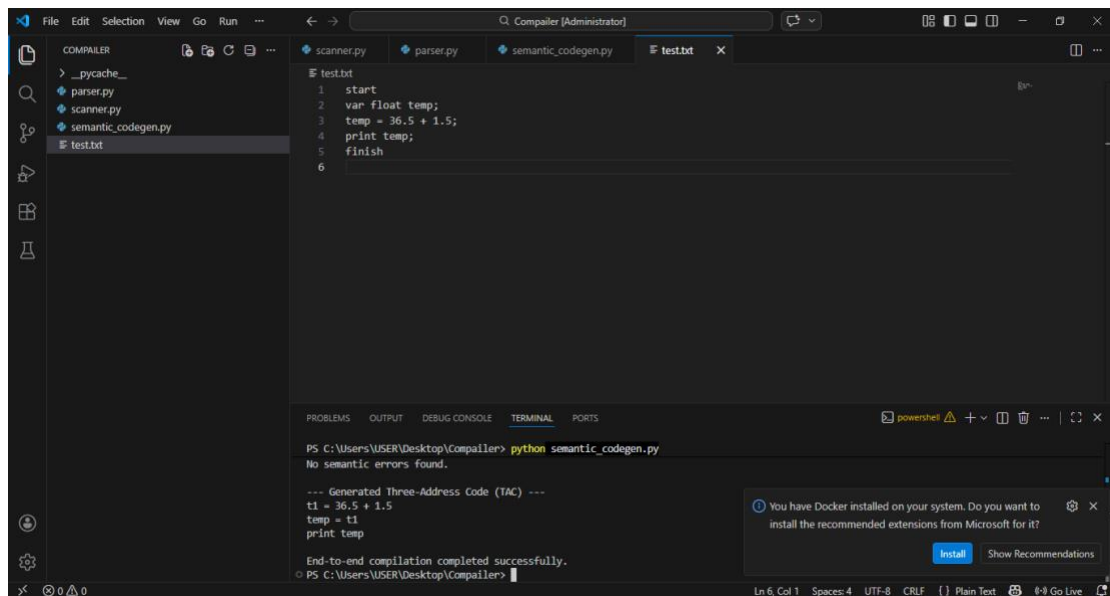
start

var float temp;

temp = 36.5 + 1.5;

print temp;

finish



The screenshot shows the Compiler IDE interface. The left sidebar displays the project structure with files: `__pycache__`, `parser.py`, `scanner.py`, `semantic_codegen.py`, and `test.txt`. The `test.txt` file is open in the editor, showing the following code:

```
1 start
2 var float temp;
3 temp = 36.5 + 1.5;
4 print temp;
5 finish
6
```

The bottom panel shows the terminal output for the command `python semantic_codegen.py`:

```
PS C:\Users\USER\Desktop\Compiler> python semantic_codegen.py
No semantic errors found.

--- Generated Three-Address Code (TAC) ---
t1 = 36.5 + 1.5
temp = t1
print temp

End-to-end compilation completed successfully.
PS C:\Users\USER\Desktop\Compiler>
```

A notification bubble in the bottom right corner states: "You have Docker installed on your system. Do you want to install the recommended extensions from Microsoft for it?" with buttons for "Install" and "Show Recommendations".

Output:

No semantic errors found.

Generated Three-Address Code (TAC):

t1 = 36.5 + 1.5

temp = t1

print temp

End-to-end compilation completed successfully.

Test Case 6: Division by Zero Error

Code:

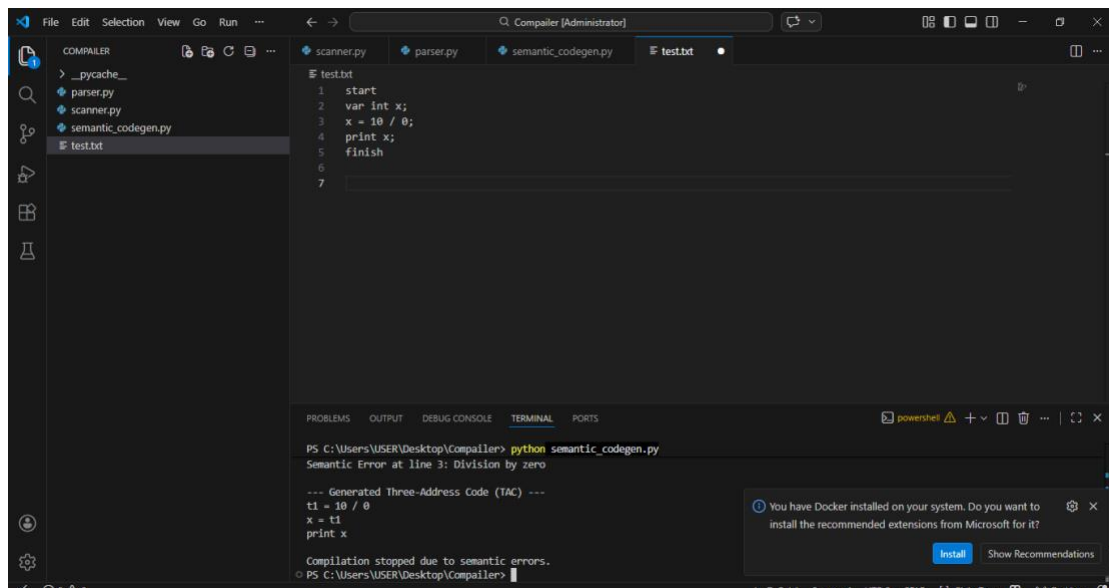
start

var int x;

x = 10 / 0;

print x;

finish



Output:

Semantic Error at line 3: Division by zero

Compilation stopped due to semantic errors.

7. End-to-End Compilation

The HealthScript compiler demonstrates a complete end-to-end compilation pipeline by processing source code through all compiler front-end phases. The process begins with lexical analysis, where the scanner reads the source program and converts it into a sequence of tokens while detecting lexical errors such as invalid symbols or malformed identifiers.

After lexical analysis, the generated token stream is passed to the syntax analyzer (parser). The parser verifies whether the token sequence conforms to the Context-Free Grammar (CFG) rules defined for the HealthScript language. During this phase, the parser checks the correctness of declarations, statements, expressions, and control structures using the Recursive Descent Parsing technique.

Once syntax analysis is completed successfully, the semantic analyzer validates the logical meaning of the program. It performs several semantic checks including variable declaration verification, duplicate declaration detection, type compatibility checking, arithmetic validation, and division-by-zero detection. The semantic analyzer also uses the enhanced symbol table to manage identifier information such as variable types, declaration lines, and scope levels.

If no semantic errors are found, the compiler proceeds to the code generation phase. In this project, Three-Address Code (TAC) is generated as an intermediate representation of the program. TAC simplifies expressions into smaller instructions that are easier to analyze and execute.

The complete compilation workflow of the HealthScript compiler is summarized as follows:

Source Code → Lexical Analysis → Syntax Analysis → Semantic Analysis → TAC Generation

The end-to-end compilation process was tested using multiple correct and incorrect HealthScript programs. Correct programs successfully generated TAC instructions, while incorrect programs produced clear lexical, syntax, or semantic error messages with line numbers.

8. Appendix

```
9. from scanner import healthscript_scanner
10.
11. class SemanticCodeGenerator:
12.     def __init__(self):
13.         self.symbol_table = {}
14.         self.semantic_errors = []
15.         self.tac = []
16.         self.temp_count = 1
17.
18.     def new_temp(self):
19.         temp = f"t{self.temp_count}"
20.         self.temp_count += 1
21.         return temp
22.
23.     def add_symbol(self, name, var_type, line):
24.         if name in self.symbol_table:
25.             self.semantic_errors.append(
26.                 f"Semantic Error at line {line}: Duplicate
27.                 declaration of variable '{name}'"
28.             )
29.         else:
30.             self.symbol_table[name] = {
31.                 "type": var_type,
```

```

32.         "line": line
33.     }
34.
35.     def get_type(self, value):
36.         if value.startswith('"') and value.endswith('"'):
37.             return "string"
38.         if value in ["true", "false"]:
39.             return "bool"
40.         try:
41.             int(value)
42.             return "int"
43.         except ValueError:
44.             pass
45.         try:
46.             float(value)
47.             return "float"
48.         except ValueError:
49.             pass
50.         if value in self.symbol_table:
51.             return self.symbol_table[value]["type"]
52.         return "unknown"
53.
54.     def check_declared(self, name, line):
55.         if name not in self.symbol_table:
56.             self.semantic_errors.append(
57.                 f"Semantic Error at line {line}: Variable
58.                 '{name}' used but not declared"
59.             )
60.             return False
61.         return True
62.
63.     def compatible(self, left_type, right_type):
64.         if left_type == right_type:
65.             return True
66.         if left_type == "float" and right_type == "int":
67.             return True
68.         return False
69.
70.     def analyze_expression(self, expr, line):
71.         parts = expr.replace(";", "").split()
72.         if len(parts) == 1:
73.             value = parts[0]

```

```

74.         if value.isidentifier() and value not in ["true",
    "false"]:
75.             self.check_declared(value, line)
76.             return self.get_type(value), value
77.
78.         if len(parts) == 3:
79.             left, op, right = parts
80.
81.             if left.isidentifier() and left not in ["true",
    "false"]:
82.                 self.check_declared(left, line)
83.             if right.isidentifier() and right not in ["true",
    "false"]:
84.                 self.check_declared(right, line)
85.
86.             left_type = self.get_type(left)
87.             right_type = self.get_type(right)
88.
89.             if op == "/" and right == "0":
90.                 self.semantic_errors.append(
91.                     f"Semantic Error at line {line}: Division by
    zero"
92.                 )
93.
94.             if op in ["+", "-", "*", "/", "%", "^"]:
95.                 if left_type not in ["int", "float"] or
    right_type not in ["int", "float"]:
96.                     self.semantic_errors.append(
97.                         f"Type Error at line {line}: Arithmetic
    operation requires numeric values"
98.                     )
99.                     result_type = "float" if "float" in [left_type,
    right_type] else "int"
100.
101.             elif op in ["==", "!=", "<", ">", "<=", ">="]:
102.                 result_type = "bool"
103.
104.             else:
105.                 result_type = "unknown"
106.
107.             temp = self.new_temp()
108.             self.tac.append(f"{temp} = {left} {op} {right}")
109.             return result_type, temp

```

```

110.
111.         return "unknown", expr
112.
113.     def process_line(self, line, line_num):
114.         line = line.strip()
115.
116.         if not line or line in ["start", "finish"]:
117.             return
118.
119.         if line.startswith("var "):
120.             parts = line.replace(";", "").split()
121.
122.             if len(parts) >= 3:
123.                 var_type = parts[1]
124.                 name = parts[2]
125.                 self.add_symbol(name, var_type, line_num)
126.
127.                 if "=" in parts:
128.                     eq_index = parts.index("=")
129.                     expr = " ".join(parts[eq_index + 1:])
130.                     expr_type, expr_value =
131.                         self.analyze_expression(expr, line_num)
132.
133.                     if not self.compatible(var_type, expr_type):
134.                         self.semantic_errors.append(
135.                             f"Type Error at line {line_num}:
136.                             Cannot assign '{expr_type}' to variable of type '{var_type}'"
137.                         )
138.                         self.tac.append(f"{name} = {expr_value}")
139.
140.                 return
141.
142.         if line.startswith("print "):
143.             expr = line.replace("print", "", 1).replace(";",
144.                 "").strip()
145.             expr_type, expr_value = self.analyze_expression(expr,
146.                 line_num)
147.             self.tac.append(f"print {expr_value}")
148.             return
149.
150.         if "=" in line and not line.startswith("if"):
151.             left, right = line.split("=", 1)
152.             name = left.strip()

```

```

149.         expr = right.replace(";", "").strip()
150.
151.         if self.check_declared(name, line_num):
152.             left_type = self.symbol_table[name]["type"]
153.             expr_type, expr_value =
self.analyze_expression(expr, line_num)
154.
155.             if not self.compatible(left_type, expr_type):
156.                 self.semantic_errors.append(
157.                     f"Type Error at line {line_num}: Cannot
assign '{expr_type}' to variable of type '{left_type}'"
158.                 )
159.
160.                 self.tac.append(f"{name} = {expr_value}")
161.
162.             return
163.
164.         if line.startswith("if "):
165.             condition = line.replace("if", "", 1).replace("then",
"".strip()
166.             cond_type, cond_value =
self.analyze_expression(condition, line_num)
167.             self.tac.append(f"if {cond_value} goto L_true")
168.             return
169.
170.         if line.startswith("while "):
171.             condition = line.replace("while", "",
1).replace("do", "").strip()
172.             cond_type, cond_value =
self.analyze_expression(condition, line_num)
173.             self.tac.append(f"while {cond_value} goto L_loop")
174.             return
175.
176.     def analyze(self, source_code):
177.         lines = source_code.splitlines()
178.
179.         for i, line in enumerate(lines, start=1):
180.             self.process_line(line, i)
181.
182.         return self.symbol_table, self.semantic_errors, self.tac
183.
184. # Main execution
185. with open("test.txt", "r") as file:

```



```

186.     source_code = file.read()
187.
188. total, tokens, sym_table, lexical_errors =
    healthscript_scanner(source_code)
189.
190. print("=== Phase 3: Semantic Analysis and Code Generation ===")
191. print()
192.
193. if lexical_errors:
194.     print("Lexical errors found:")
195.     for err in lexical_errors:
196.         print(err)
197.
198. else:
199.     analyzer = SemanticCodeGenerator()
200.     symbol_table, semantic_errors, tac =
        analyzer.analyze(source_code)
201.
202.     print("--- Enhanced Symbol Table ---")
203.     for name, info in symbol_table.items():
204.         print(
205.             f"{name} | Type: {info['type']} | Scope:
        {info['scope']} | Declared at line: {info['line']}"
206.         )
207.
208.     print()
209.     print("--- Semantic Errors ---")
210.     if semantic_errors:
211.         for err in semantic_errors:
212.             print(err)
213.     else:
214.         print("No semantic errors found.")
215.
216.     print()
217.     print("--- Generated Three-Address Code (TAC) ---")
218.     for instruction in tac:
219.         print(instruction)
220.
221.     print()
222.     if not semantic_errors:
223.         print("End-to-end compilation completed successfully.")
224.     else:
225.         print("Compilation stopped due to semantic errors.")

```

9. Conclusion

Phase 3 successfully extends the HealthScript compiler by adding semantic analysis and intermediate code generation. The semantic analyzer validates program meaning by checking declarations, type compatibility, duplicate identifiers, and division by zero. The code generator produces readable Three-Address Code for correct programs. This phase completes the main compiler front-end pipeline and demonstrates the integration of lexical, syntax, and semantic analysis.

References

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. *Compilers: Principles, Techniques, and Tools (2nd Edition)*. Pearson Education, 2006.
2. Python Software Foundation. *Python 3 Documentation*. Available at: <https://docs.python.org/3/>
3. GeeksforGeeks. *Recursive Descent Parser in Compiler Design*. Available at: <https://www.geeksforgeeks.org/recursive-descent-parser/>
4. GeeksforGeeks. *FIRST and FOLLOW in Compiler Design*. Available at: <https://www.geeksforgeeks.org/first-set-in-syntax-analysis/>
5. LLVM Project Documentation. Available at: <https://llvm.org/docs/>
6. Friedl, Jeffrey E. F. *Mastering Regular Expressions*. O'Reilly Media, 2006.
7. Wikipedia Contributors. *Three-address code*. Available at: https://en.wikipedia.org/wiki/Three-address_code

We Also Used Chatgpt to help us with the report, and Blackbox to help us enhance our Code.